

MLOps Experimental Learning Assignment

End-to-End ML Model Development, CI/CD, Deployment & Monitoring

Course: MLOps (S1-25_AIMLCZG523)

Assignment: Experimental Learning – Assignment I

Dataset: Heart Disease UCI Dataset

Student Name:

Name	ID	Participation
Manvenddra Singh	2024aa05568	100
Rohit Prabhakarn	2024aa05706	100
Adithya D Sheno	2024aa05176	100
Hitesh Padname	2024aa05180	100
Trilok Kumar	2024aa05373	100

Introduction

1. Introduction

Machine Learning systems in real-world environments require more than just model training. They demand automation, reproducibility, continuous integration, deployment, and monitoring. This assignment focuses on implementing an **end-to-end MLOps pipeline** that mirrors production-grade machine learning systems.

The given problem is **Heart Disease Risk Prediction**, a classical supervised learning problem where patient health attributes are used to predict the presence or absence of heart disease. The solution is designed to be scalable, reproducible, containerized, and deployable using modern DevOps and MLOps practices.

The assignment covers the complete lifecycle from data ingestion and experimentation to CI/CD, Dockerization, Kubernetes deployment, and monitoring.

Dataset Description

2. Dataset Description

- **Source:** UCI Machine Learning Repository
- **Dataset Name:** Heart Disease Dataset
- **Format:** CSV
- **Number of Features:** 14+
- **Target Variable:** Binary (Presence / Absence of heart disease)

2.1 Feature Overview

The dataset includes patient health indicators such as:

- Age
- Gender
- Chest pain type
- Blood pressure
- Cholesterol levels
- Resting ECG results
- Maximum heart rate
- Exercise-induced angina
- ST depression
- Thalassemia status

A data download script and preprocessing logic were implemented to ensure repeatability.

```
# Reading the dataset from a CSV file into a Pandas DataFrame
#Sanity check of data loaded
file_path = r"/Users/hades/Documents/BITS Lectures/Semester 3/MLOPS/Assignment/heart_disease_uci.csv"

# Read the CSV file into a DataFrame
df = pd.read_csv(file_path)
# Displaying the first 5 rows of the dataset to get a quick overview of the data
df.head()
```

Python

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

provide a statistical summary of the dataset using describe() and displays the number of rows and columns using shape. It helps in understanding the data distribution and structure.

Data Acquisition & EDA

3. Data Acquisition & Exploratory Data Analysis

Data preprocessing and EDA were performed to understand feature distributions and relationships.

3.1 Data Cleaning

- Missing values handled using **SimpleImputer**
- Categorical features encoded using **OneHotEncoder**
- Numerical features scaled using **StandardScaler**

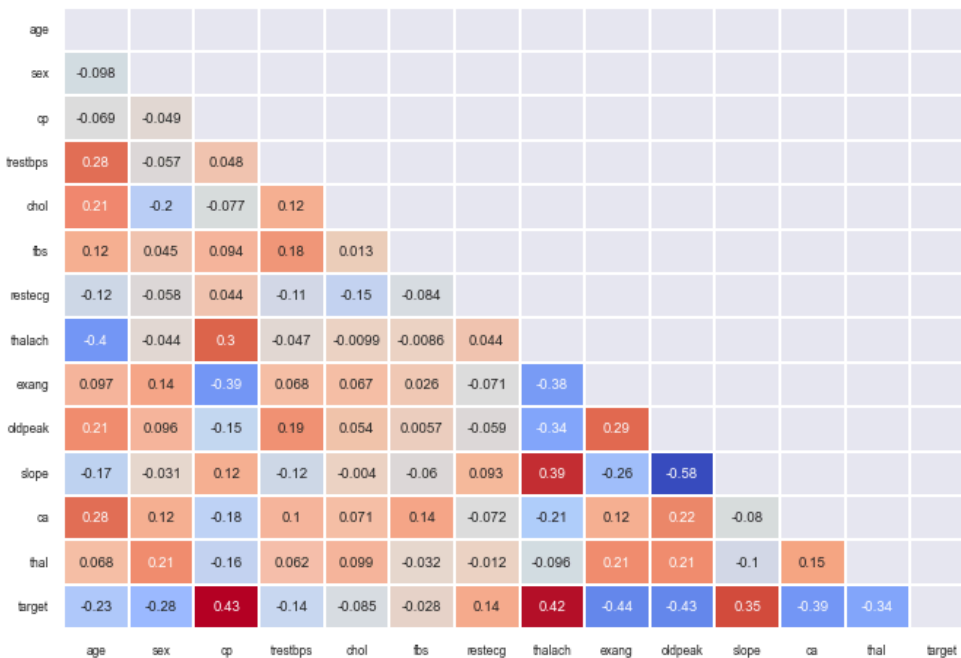
3.2 Exploratory Data Analysis

- Feature distribution plots
- Correlation heatmap
- Class balance analysis

The insights from EDA guided feature engineering and model selection.

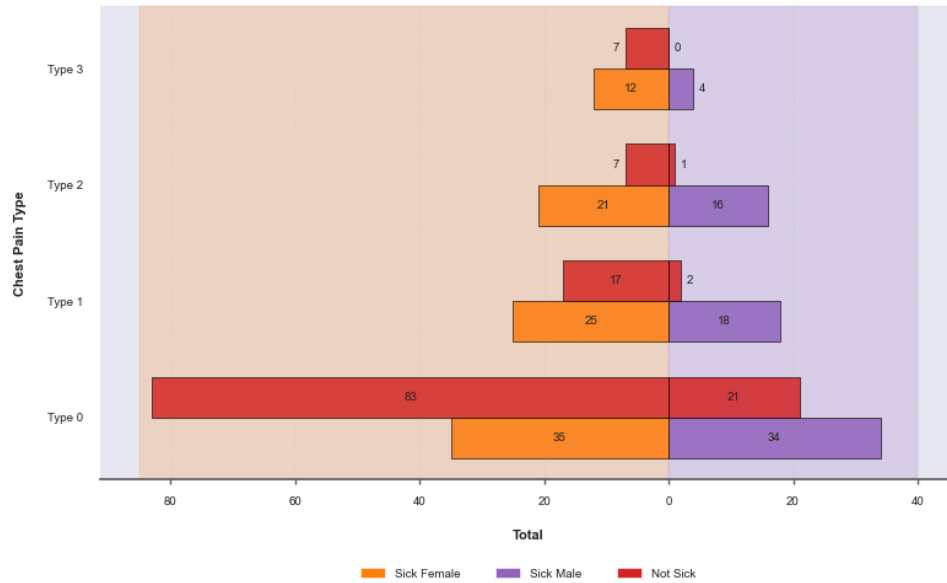
Numerical Variables Correlation Map

Chest pain type, max heart rate, and slope positively correlate with target variables.



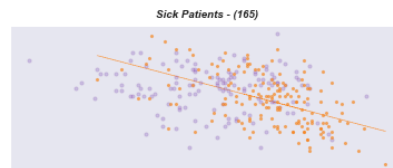
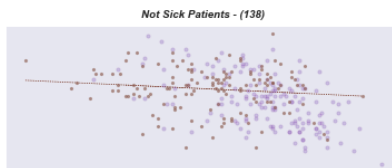
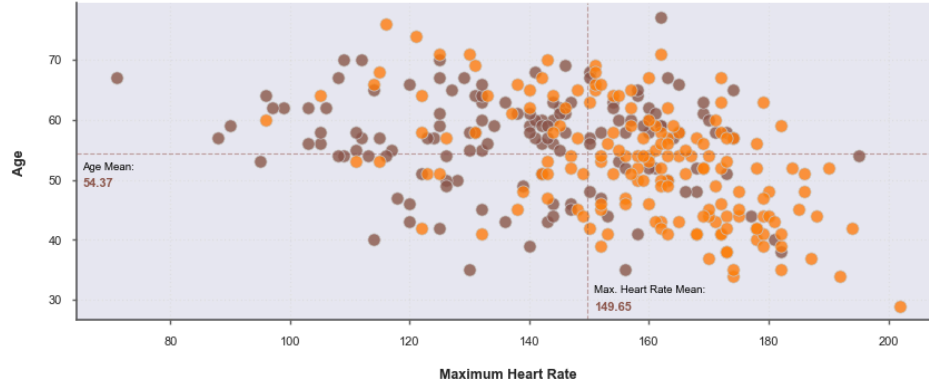
Disease Distribution based on Chest Pain Type in Each Gender

Chest pain types 1, 2, and 3 **have more sick patients** than those **who don't**.

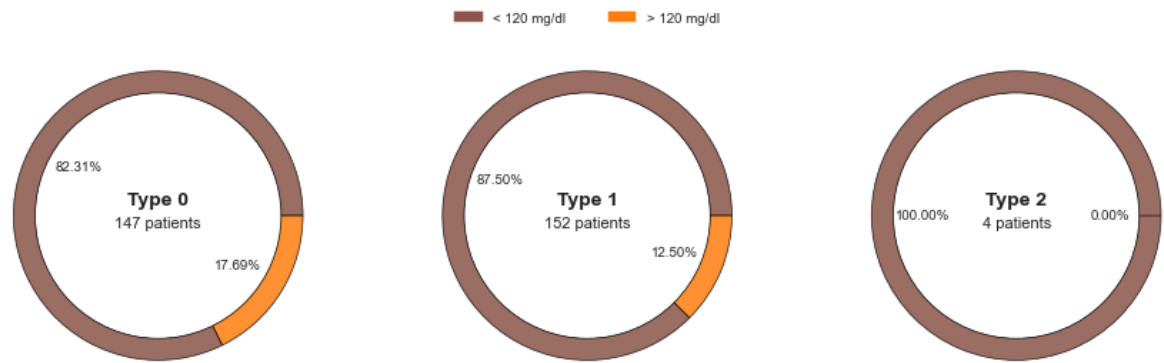


Maximum Heart Rate vs. Age based on Patients Sickness

Patients who tend to get heart disease are **less than 54 years old** and have **max. heart rate over 149**.



Fasting Blood Sugar Distribution by Resting Electrocardiographic Results

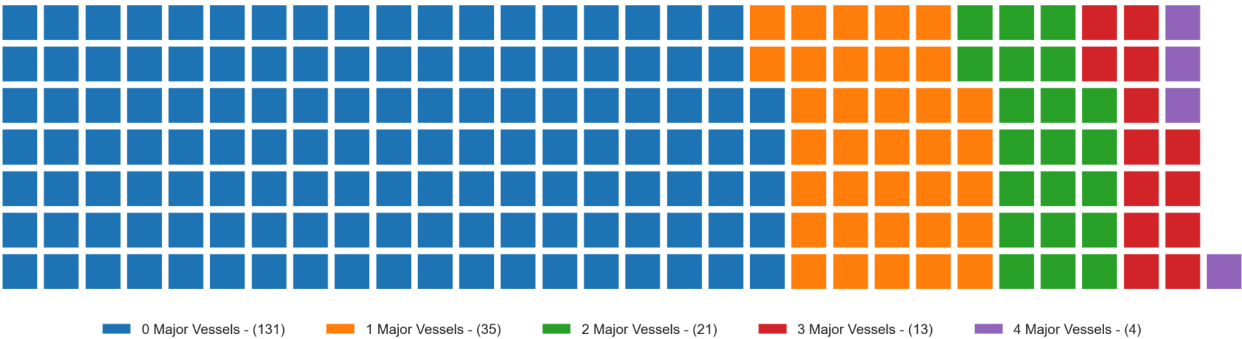


Resting electrocardiograph type 0 and 1 have higher distribution compared to type 2.
Only type 0 and 1 have patients with fasting blood sugar over 120 mg/dl, while type 2 does not

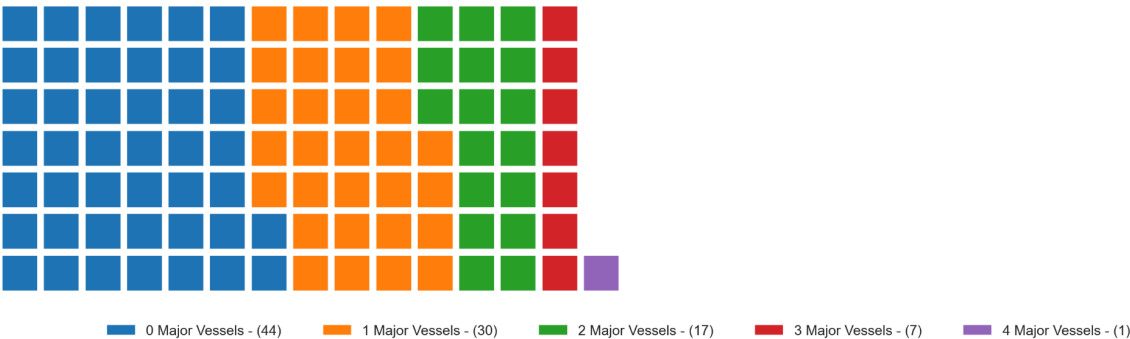
Number of Major Vessles Distribution based on Exercise Induced Angina

The major vessel distribution proportion in patients with and without exercise-induced angina is almost the same.

Don't Have Exercise Induced Angina

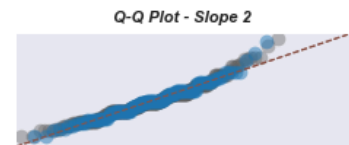
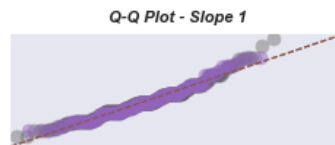
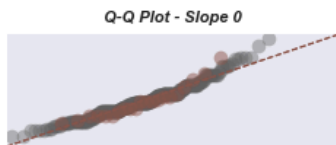
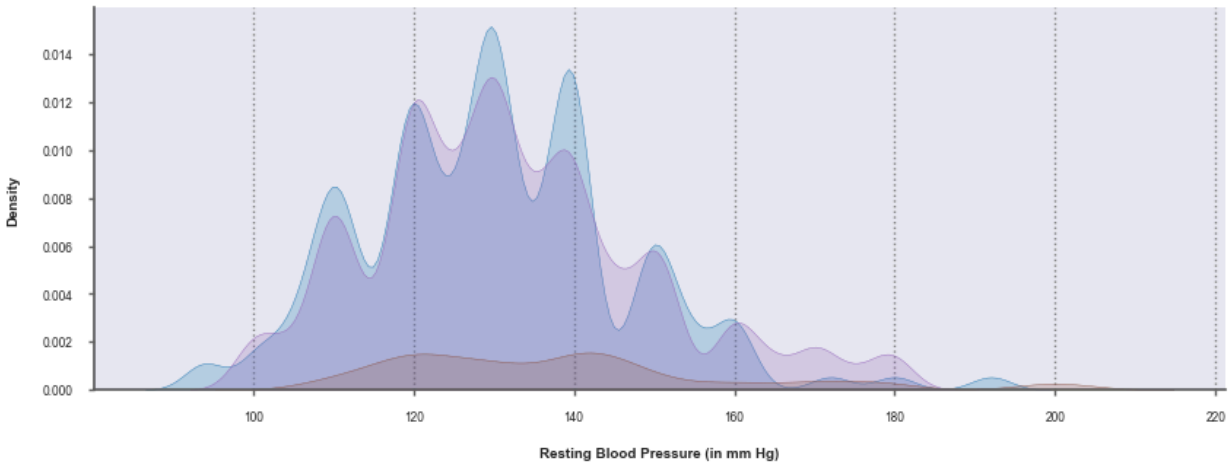


Have Exercise Induced Angina



Resting Blood Pressure Distribution based on Slope

Each slope type distribution is **moderately right-skewed**.



Feature Engineering & Model Development

4. Feature Engineering & Model Development

A unified preprocessing pipeline was built using **ColumnTransformer** to ensure consistency between training and inference.

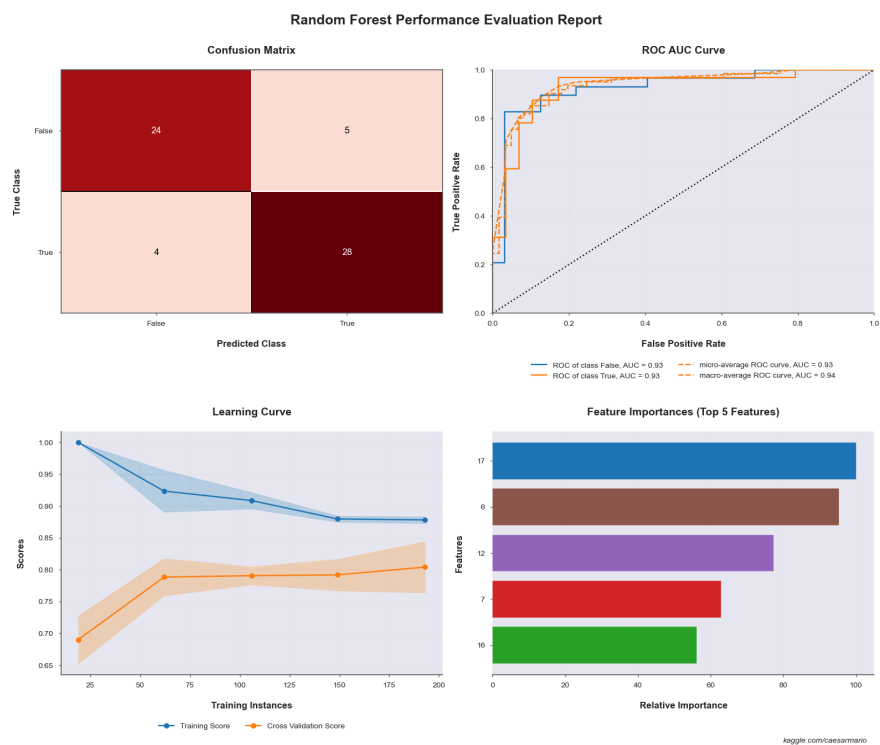
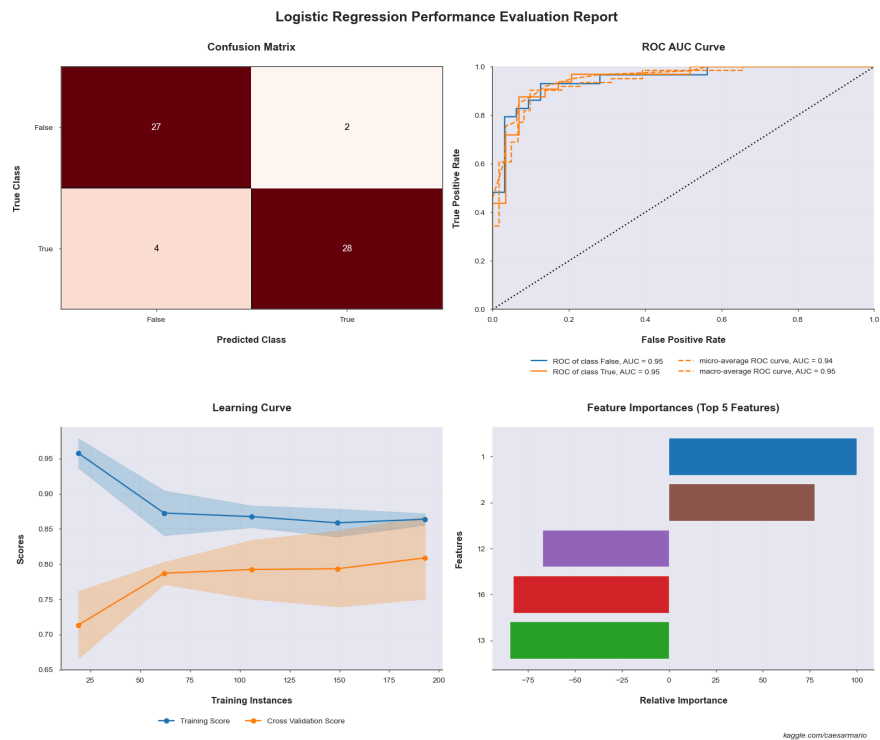
4.1 Models Trained

1. Logistic Regression
2. Random Forest Classifier

4.2 Model Evaluation Metrics

- Accuracy
- Precision
- Recall
- ROC-AUC

Cross-validation was used to evaluate performance robustness. The Random Forest model achieved superior performance and was selected as the final model.



Experiment Tracking (MLflow)

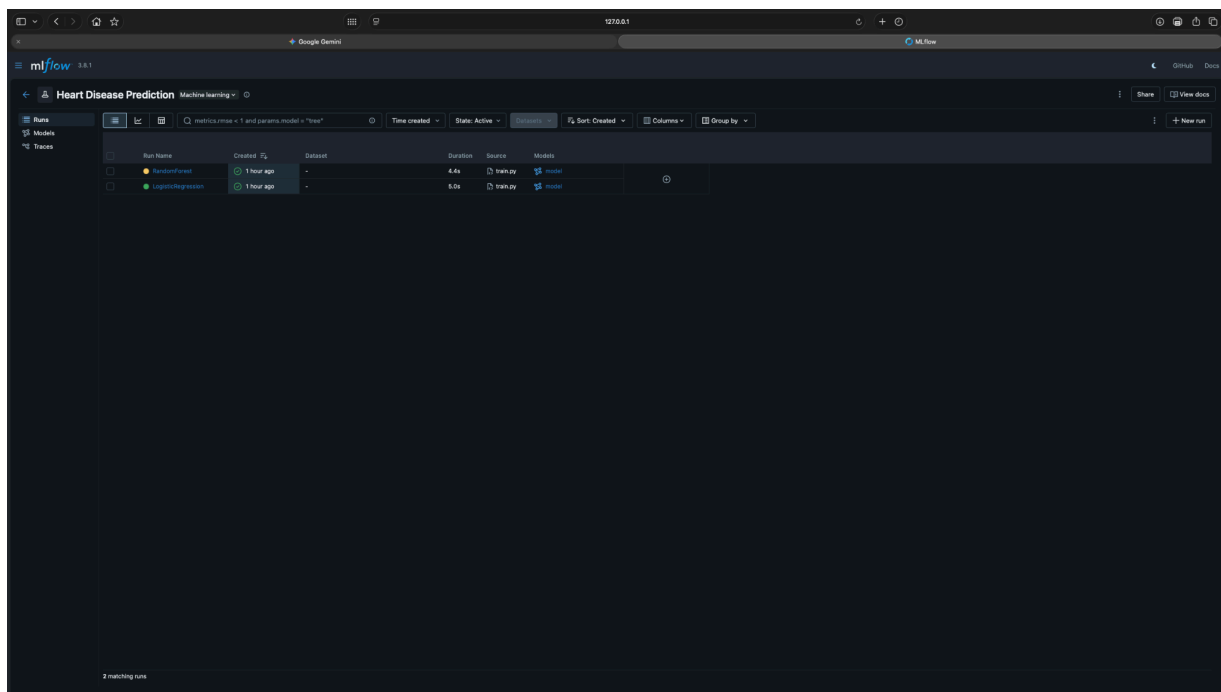
5. Experiment Tracking

MLflow was integrated to track experiments and artifacts.

5.1 Logged Information

- Model parameters
- Evaluation metrics
- Training artifacts
- Preprocessing pipeline
- Visualizations

Each experiment run was tracked under a named experiment, enabling easy comparison and reproducibility.



mi/flow 3.8.1

Heart Disease Prediction > Models > model Not registered

Overview Traces Artifacts

MLmodel

conda.yaml

model.pkl

python_env.yaml

requirements.txt

MLmodel 7308

Path: /Users/hades/Documents/BIT5 Lectures/Semester 3/ML OPS/Assignment/heart-mlops-refactor/hub/m-2b0cedeb974a1b847f4a070a797be/artifacts/ml_model

artifact_path: /Users/hades/Documents/BIT5 Lectures/Semester 3/ML OPS/Assignment/heart-mlops-refactor/ml_runs/1/models/m-2b0cedeb974a1b847f4a070a797be/artifacts

flows:

python_function:

env:

conda: conda.yaml

virtualenv: python_env.yaml

loader_module: mlflow.sklearn

model_path: model.pkl

predict_fn: predict

python_version: 3.12.7

sklearn:

code: null

pickled_model: model.pkl

serialization_format: Cloudpickle

sklearn_version: 1.4.0

mlflow_version: 3.8.1

model_size: m-2b0cedeb974a1b847f4a070a797be

model_size_bytes: 125799

model_uuid: m-2b0cedeb974a1b847f4a070a797be

prompt: null

run_id: 8759a27b8ba47eeeb4884f8849

utc_time_created: '2025-01-05 18:42:28.680937'

mi/flow 3.8.1

Heart Disease Prediction > Models > model Not registered

Overview Traces Artifacts

Register model

Description

No description

Metrics (8)

Search metrics

Metric	Dataset	Source run	Value
cv_best_model_auc	-	RandomForest	0.9133109367484...
train_accuracy	-	RandomForest	0.97265972697369...
train_precision	-	RandomForest	0.9712320218827...
train_recall	-	RandomForest	0.9712320218827...
train_f1	-	RandomForest	0.9712320218827...
train_auc_auc	-	RandomForest	0.9985962449552...

Parameters (8)

Search parameters

Parameter	Value
best_model_max_depth	10
best_model_min_samples_split	5
best_model_estimators	200
cv_splits	5
model	RandomForest
random_seed	42

Runs

Search runs

Run	Input
RandomForest	-

About this logged model

Created at 1 hour ago

Created by hades

Status Ready

Model ID m-2b0cedeb974a1b84...

Source run RandomForest

Source run ID 875a27b8ba47eeeb4884f8849

Logged from train.py

Datasets used

None

Model versions

None

Model Packaging & Reproducibility

6. Model Packaging & Reproducibility

The final trained model, along with preprocessing steps, was serialized using **joblib**.

6.1 Reproducibility Measures

- Exact dependency versions pinned in requirements.txt
- Consistent Python version (3.10)
- Model was exported in **.pkl**
- Model packaged inside Docker image

This ensured identical inference behavior across environments.

CI/CD Pipeline & Automated Testing

7. CI/CD Pipeline

A CI/CD pipeline was implemented using **GitHub Actions**.

7.1 Pipeline Stages

- Code linting using **flake8**
- Unit testing using **pytest**
- Automated training execution

7.2 Testing Strategy

Unit tests were written for:

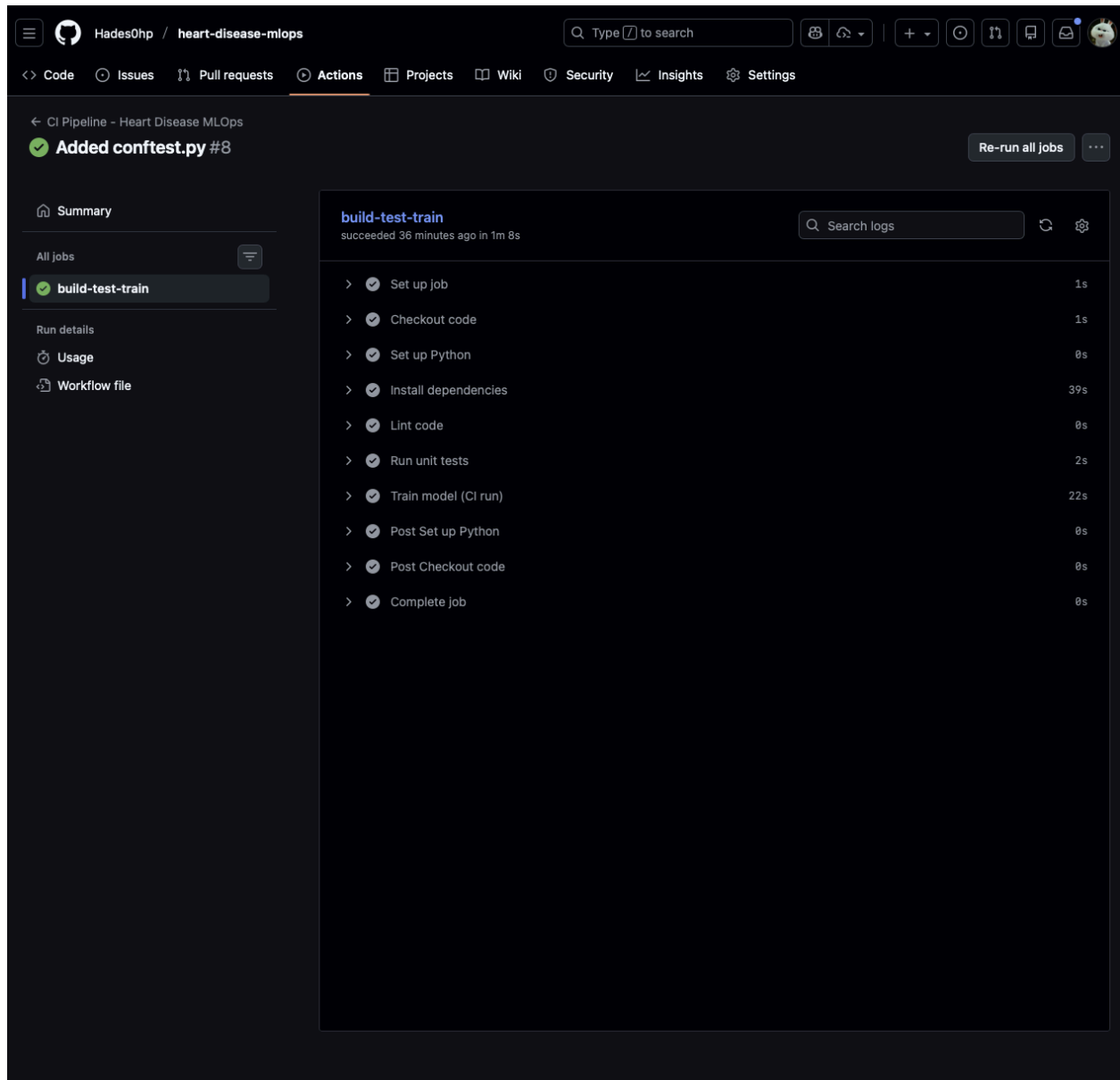
- Data loading
- Feature engineering
- Training pipeline

The pipeline fails automatically on code quality or test failures.

🔍 Filter workflow runs

Showing runs from all workflows

Today at 8:48 PM ...
44s



Model Containerization

8. Model Containerization

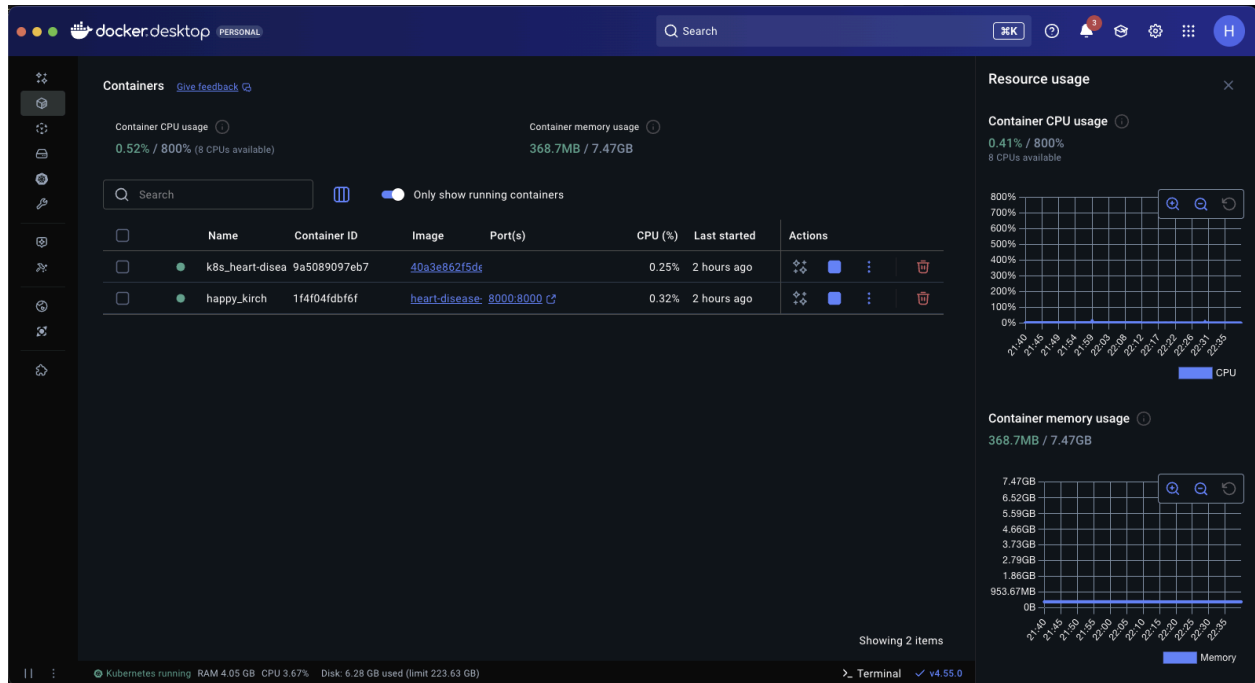
The inference service was containerized using **Docker**.

8.1 API Implementation

- Framework: **FastAPI**
- Endpoint: /predict
- Input: JSON

- Output: Prediction + confidence score
- Swagger UI enabled for testing

The Docker image includes the trained model, dependencies, and API code.



Production Deployment (Kubernetes)

9. Production Deployment

The Dockerized API was deployed to **Kubernetes** using local orchestration (Docker Desktop)

9.1 Kubernetes Resources

- Deployment for managing pods
- NodePort Service for external access

The API was accessed successfully via the Kubernetes service endpoint.

Ask Gordon

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

Docker Hub

Docker Scout

Extensions

Kubernetes

Cluster

Cluster

Active

Cluster type

kubeadm

Nodes

1

Kubernetes version

v1.34.1

Deployments

Name	Status	Pods
heart-disease-api	Available	1/1

Nodes

Name	Status
docker-desktop	Ready

Ingresses

Pods

Name	Status
heart-disease-api-67c45dd78-14w26	Running

Services

Name	Cluster IP	Ports
heart-disease-service	10.97.210.197	80/TCP
kubernetes	10.96.0.1	443/TCP

Config maps and secrets

Stop

Edit cluster

Engine running

Kubernetes running

RAM 4.05 GB

CPU 7.44%

Disk 6.38 GB used (limit 223.63 GB)

Terminal

v4.95.0

Heart Disease Prediction API

0.1.0 OAS 3.1

[/openapi.json](#)

default



GET / Health

Parameters Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema "string"	No links

POST /predict Predict

Parameters Cancel Reset

No parameters

Request body required

application/json

Edit Value Schema

Page 10 — Monitoring, Logging & Conclusion

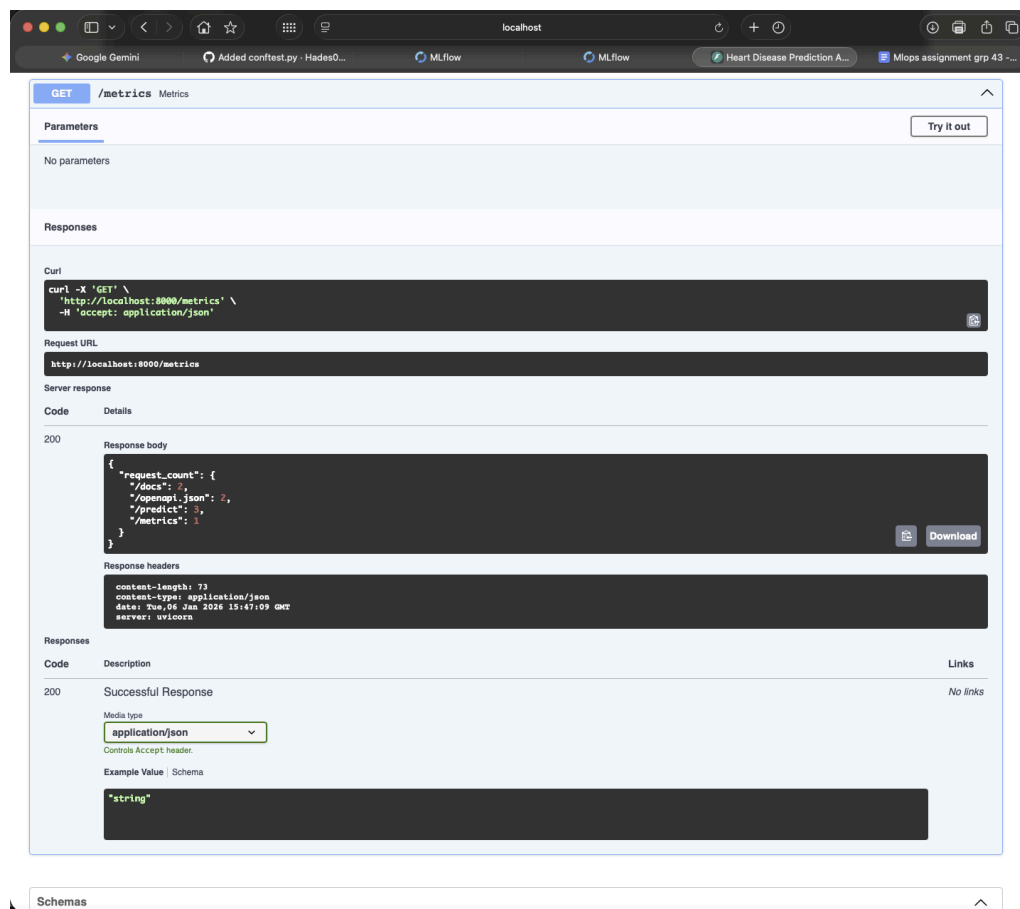
10. Monitoring & Logging

10.1 Logging

- FastAPI middleware logs:
 - HTTP method
 - Endpoint
 - Status code
 - Request latency

10.2 Monitoring

- /metrics endpoint exposing request counts
- Logs verified locally and in Kubernetes



Conclusion

This assignment successfully demonstrates a complete MLOps lifecycle—from data ingestion and experimentation to CI/CD, containerization, Kubernetes deployment, and monitoring. By following best practices in reproducibility, automation, and observability, the solution closely mirrors real-world production ML systems.
